Pr(*A*) is a function that accepts an action-value vector (array) and returns a probability distribution over the actions, such that higher value actions have higher probabilities. For example, if your action-value array has four possible actions and they all currently have the same value, say `A = [10, 10, 10, 10]`, then `Pr(A) = [0.25, 0.25, 0.25, 0.25]`. In other words, all the probabilities are the same and must sum to 1.

The numerator of the fraction exponentiates the action-value array divided by a parameter, τ, yielding a vector of the same size (i.e., length) as the input. The denominator sums over the exponentiation of each individual action value divided by τ, yielding a single number.

τ is a parameter called *temperature* that scales the probability distribution of actions. A high temperature will cause the probabilities to be very similar, whereas a low temperature will exaggerate differences in probabilities between actions. Selecting a value for this parameter requires an educated guess and some trial and error. The mathematical exponential e^x is a function call to `np.exp(...)` in numpy. It will apply the function element-wise across the input vector. Here's how we actually write the softmax function in Python.

Listing 2.7. The softmax function

```
def softmax(av, tau=1.12):
    softm = np.exp(av / tau) / np.sum( np.exp(av / tau) )
    return softm
```

When we implement the previous 10-armed bandit problem using `softmax`, we don't need the `get_best_arm` function anymore. Since `softmax` produces a weighted probability distribution across our possible actions, we can randomly select actions according to their relative probabilities. That is, our best action will get chosen more often because it will have the highest `softmax` probability, but other actions will be chosen at lower frequencies.

To implement this, all we need to do is apply the `softmax` function over the second column (column index 1) of the `record` array, since that's the column that stores the current mean reward (the action values) for each